

ALU Verification

using a Class-Based Environment in SystemVerilog

Verification Report

By Ahmed H. Ashour

ALU Design File

To verify this ALU, we will need to check that each opcode produces the correct result, we should confirm that the outputs match expected values across directed edge cases (like overflow or zero results) and randomized inputs.

```
alu.v  
  
module alu (  
    input [3:0] a, b,  
    input [1:0] opcode,  
    output reg [3:0] result  
);  
    always @(C*) begin  
        case (opcode)  
            2'b00: result = a + b; // add  
            2'b01: result = a - b; // sub  
            2'b10: result = a & b; // and  
            2'b11: result = a ^ b; // xor  
            default: result = 4'b0;  
        endcase  
    end  
endmodule
```

Figure 1 : ALU Code

Environment sttructure

This testbench structure follows a layered verification model: the **test** runs the **env**, which contains the **agent** and the **scoreboard**. The **agent** contains the **generator**, **driver**, and **monitor** creating transactions and driving them to the DUT through the **interface**, and observed. Finally, the **scoreboard** compares monitored results against expected values.

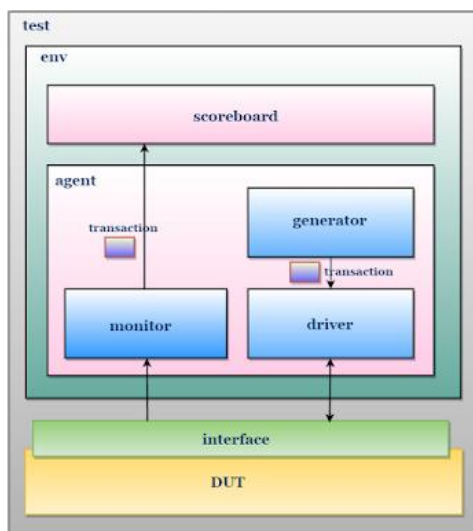


Figure 2 : Environment Structure

Transaction Class

This transaction class defines the randomized inputs a, b, and opcode along with the output field result for ALU verification. It also includes a display function to print transaction details.

```
transaction.sv

class transaction;
  randc bit [3:0] a;
  randc bit [3:0] b;
  rand bit [1:0] opcode;
  bit [3:0] result;

  function void display(input string name = "transaction");
    $display("%s: a=%0d, b=%0d, opcode=%0b", name, a, b, opcode);
  endfunction
endclass
```

Figure 3 : Transaction Class

Generator Class

This generator class is responsible for creating transactions starting with a couple of directed edge case tests and then producing randomized tests. It sends each transaction into its mailbox and synchronizes with the driver using the gen_handover event.

```
generator.sv

class generator;
  transaction t_gen;
  int iterations;
  mailbox gen_mail;
  event gen_handover;

  function new();
    this.gen_mail = new();
  endfunction

  task run_generator();
    t_gen = new();
    iterations = 100; // number of generated transactions
    for(int i = 1; i ≤ iterations; i++) begin
      //direct tests
      if(i == 1) begin
        t_gen.a = 4'd15;
        t_gen.b = 4'd15;
        t_gen.opcode = 2'b00; //add 15+15
        gen_mail.put(t_gen);
        @(gen_handover);
      end

      else if(i == 2) begin
        t_gen.a = 4'd0;
        t_gen.b = 4'd15;
        t_gen.opcode = 2'b01; //subtract 0-15
        gen_mail.put(t_gen);
        @(gen_handover);
      end

      //random tests
      else begin
        assert(t_gen.randomize()) else $fatal("randomize failed");
        gen_mail.put(t_gen);
        @(gen_handover);
      end
    end
  endtask
endclass
```

Figure 4 : Generator Class

Driver Class

This driver class gets transactions from its mailbox then applies a short delay and then drives the ALU inputs through the virtual interface. After loading the transaction into the DUT, it triggers the `driv_handover` event so the generator can synchronize.

```
drivers.v  
  
class driver;  
transaction t_driv;  
mailbox driv_mail;  
event driv_handover;  
virtual intf driv_intf;  
  
function new();  
    this.driv_mail = new();  
endfunction  
  
task run_driver();  
  
    forever begin  
        t_driv = new();  
        driv_mail.get(t_driv);  
        #5; //delay between drives since there is no clk  
        t_driv.display("driver");  
        //load transaction into interface  
        driv_intf.a = t_driv.a;  
        driv_intf.b = t_driv.b;  
        driv_intf.opcode = t_driv.opcode;  
  
        ->driv_handover;  
    end  
endtask  
endclass
```

Figure 5 : Driver Class

Monitor Class

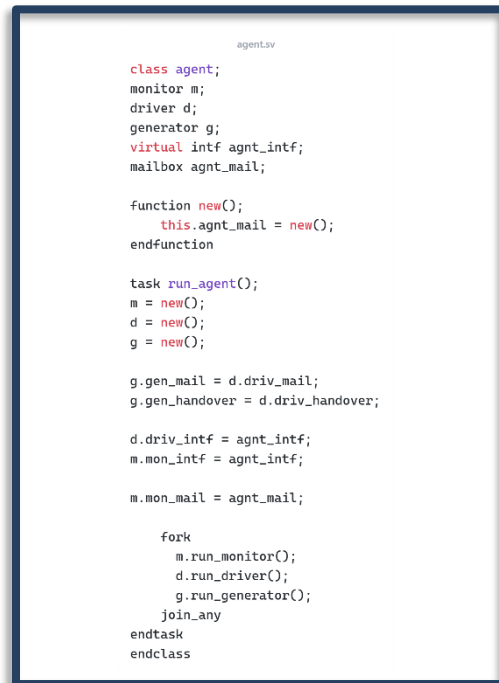
This monitor class observes the DUT through the virtual interface, sampling inputs and outputs after a short delay to ensure the driver has already applied values. It packages the sampled data into a transaction, places it in the mailbox for the scoreboard and prints its details.

```
class monitor;  
transaction t_mon;  
mailbox mon_mail;  
virtual intf mon_intf;  
  
function new();  
    this.mon_mail = new();  
endfunction  
  
task run_monitor();  
    t_mon = new();  
    #1; //delay to allow driver to drive the interface and the monitor to sample it after #1  
    forever begin  
        #5; //delay between transactions  
        t_mon.a = mon_intf.a;  
        t_mon.b = mon_intf.b;  
        t_mon.opcode = mon_intf.opcode;  
        t_mon.result = mon_intf.result;  
        mon_mail.put(t_mon);  
        t_mon.display("monitor");  
    end  
endtask  
endclass
```

Figure 6 : Monitor Class

Agent Class

This agent class contains the **generator**, **driver**, and **monitor**, wiring their mailboxes and interface connections. It constructs each component and connects the generators mailbox directly to the drivers mailbox and links the monitor to the agent mailbox and then runs all three concurrently to drive the observe and check the DUT.

The image shows a screenshot of a text editor window titled 'agent.sv'. It contains Verilog code for an 'agent' class. The code defines a 'monitor' m, a 'driver' d, and a 'generator' g. It includes a 'virtual' interface 'agnt_intf' and a mailbox 'agnt_mail'. A 'new()' function initializes 'this.agnt_mail = new()'. A 'run_agent()' task creates instances of m, d, and g, connects their mailboxes and interfaces, and forks them to run concurrently. The code ends with 'endtask' and 'endclass'.

```
agent.sv

class agent;
  monitor m;
  driver d;
  generator g;
  virtual intf agnt_intf;
  mailbox agnt_mail;

  function new();
    this.agnt_mail = new();
  endfunction

  task run_agent();
    m = new();
    d = new();
    g = new();

    g.gen_mail = d.driv_mail;
    g.gen_handover = d.driv_handover;

    d.driv_intf = agnt_intf;
    m.mon_intf = agnt_intf;

    m.mon_mail = agnt_mail;

    fork
      m.run_monitor();
      d.run_driver();
      g.run_generator();
    join_any
  endtask
endclass
```

Figure 7 : Agent Class

Scoreboard Class

This scoreboard class checks the DUT's output against a golden model by calculating the expected result for each opcode. It tracks how many tests pass or fail and provides a summary of verification results at the end.

```

class scoreboard;

transaction t_score;
mailbox score_mail;
int passed_test_count;
int failed_test_count;

function new();
    this.score_mail = new();
    passed_test_count = 0;
    failed_test_count = 0;
endfunction

function void display_results();
    $display("scoreboard: passed tests: %0d, failed tests: %0d", passed_test_count, failed_test_count);
endfunction

task run_scoreboard();
    bit [3:0] golden_result;

    forever begin
        t_score = new();
        score_mail.get(t_score);
        t_score.display("scoreboard");

        case(t_score.opcode)
            2'b00: golden_result = t_score.a + t_score.b;
            2'b01: golden_result = t_score.a - t_score.b;
            2'b10: golden_result = t_score.a & t_score.b;
            2'b11: golden_result = t_score.a ^ t_score.b;
            default: golden_result = 4'b0;
        endcase

        if (golden_result == t_score.result) begin
            failed_test_count++;
            $display("scoreboard: test failed: a=%0d b=%0d opcode=%0b expected=%0d got=%0d",
                t_score.a, t_score.b, t_score.opcode, golden_result, t_score.result);
        end
        else begin
            passed_test_count++;
            $display("scoreboard: test passed: a=%0d b=%0d opcode=%0b result=%0d",
                t_score.a, t_score.b, t_score.opcode, t_score.result);
        end
    end
endtask
endclass

```

Figure 8 : Scoreboard Class

Environment Class

This environment class is the **top level container** that contains agent and scoreboard together. It connects the agent's interface to the environment interface and routes the monitor's mailbox into the scoreboard.

```

class enviroment;
    agent a;
    scoreboard sb;
    virtual intf env_intf;

    function new(virtual intf vif);
        this.env_intf = vif;
    endfunction

    task run_enviroment();
        a = new();
        sb = new();

        a.agnt_intf = env_intf;
        sb.score_mail = a.agnt_mail;

        fork
            a.run_agent();
            sb.run_scoreboard();
        join_any

    endtask
endclass

```

Figure 9 : Environment Class

Test

This test module contains everything together it instantiates the DUT (alu) and the verification environment and the shared interface. In the initial block the environment is created then the agent and scoreboard are run concurrently to generate, drive, monitor, and check transactions. Then after a short delay to capture the last sample, the scoreboard prints the pass/fail summary before stopping the simulation.

```
test.sv

`timescale 1ns/1ps

`include "transaction.sv"
`include "generator.sv"
`include "monitor.sv"
`include "driver.sv"
`include "agent.sv"
`include "scoreboard.sv"
`include "environment.sv"
`include "interface.sv"

module test;
  intf alu_intf();
  environment e;
  alu dut (.a(alu_intf.a),.b(alu_intf.b),.opcode(alu_intf.opcode),.result(alu_intf.result));

  initial begin
    e = new(alu_intf);
    e.run_environment();
    #2; // for the monitor to capture the last transaction
    e.sb.display_results();
    $stop;
  end
endmodule
```

Figure 10 : Test Code

Simulation Results

the simulation completed 100 transactions with zero failures verifying that the ALU results for directed and randomized case. The scoreboard summary confirms the verification environment is working properly with all tests passing and no mismatches detected.

```
# monitor: a=5, b=1, opcode=10
# scoreboard: a=5, b=1, opcode=10
# scoreboard: test passed: a=5 b=1 opcode=10 result=1
# driver: a=8, b=6, opcode=1
# monitor: a=8, b=6, opcode=1
# scoreboard: a=8, b=6, opcode=1
# scoreboard: test passed: a=8 b=6 opcode=1 result=2
# driver: a=4, b=9, opcode=0
# monitor: a=4, b=9, opcode=0
# scoreboard: a=4, b=9, opcode=0
# scoreboard: test passed: a=4 b=9 opcode=0 result=13
# scoreboard: passed tests: 100, failed tests: 0
```

Figure 11 : Simulation Results